Università di Bologna

Dipartimento di Informatica - Scienza e Ingegneria

Corso di Laurea Magistrale in Informatica

Relazione sul Progetto di Intelligenza Artificiale del gruppo Sdrogondo Gongo AI

Addestramento di un Agente Autonomo per la Guida su Pista con Deep Q-Network (DQN)

Gruppo: Sdrogondo Gongo AI
Studenti:

Lorenzo **Davalli**, Emiliano **Chiarini**, Luca **Di Mauro**, Lorenzo **Pirchio**

Anno Accademico 2025/2026

Indice

1 Introduzione **3**
1.1 Obiettivo del Progetto . . . 3
1.2 Tecnologie Utilizzate . . . 3

2 Background Teorico **3**
2.1 Reinforcement Learning . . . 3
2.2 Deep Q-Learning . . . 4
2.3 Processo di Decisione di Markov (MDP) . . . 5

3 Struttura del progetto **5**

4 L'Ambiente: TrackEnv **6**
4.1 Descrizione Generale . . . 6
4.2 Rappresentazione del circuito . . . 6
4.3 Generazione Matrice Distanze . . . 6
4.4 Generazione Matrice Distanza Centro . . . 7
4.5 Spazio delle Osservazioni . . . 8
4.6 Spazio delle azioni . . . 8
4.6.1 Modalità "velocity" (Accelerazione) . . . 8
4.6.2 Modalità "simple" (Movimento Ridotto) . . . 9
4.7 Checkpoints . . . 10
4.8 Sistema di Ricompense . . . 10
4.9 Rendering con PyGame . . . 11

5 L'Agente: Deep Q-Network **11**
5.1 Architettura della Rete Neurale . . . 11
5.2 Experience Replay . . . 12
5.3 Iperparametri dell'Agente . . . 13
5.4 Strategia Epsilon-Greedy . . . 13
5.5 Algoritmo di Apprendimento . . . 14

6 Pipeline di Training **15**
6.1 Flusso di esecuzione . . . 15
6.2 Parametri di Training . . . 16

7 Testing e Valutazione **16**
7.1 Modalità Test . . . 16
7.2 Metriche di Valutazione . . . 16

8 Circuiti Disponibili **17**
8.1 Altri tracciati . . . 18

9 Colonna Sonora **18**

10 Strumenti di Analisi **18**
10.1 Grafico Training . . . 18
10.2 Heatmap . . . 19
10.3 Report PDF . . . 19

11 Ottimizzazioni **19**

12 Grafica del Tracciato **20**

13 Risultati **21**
13.1 Esperimento 1: Agente Base (Movimento Discreto) 21
13.2 Esperimento 2: Ambiente Migliorato 22
13.3 Esperimento 3: Agente con Accelerazione 24

14 Conclusioni e lavori futuri **26**

15 Repository **28**

1 Introduzione

1.1 Obiettivo del Progetto

Il progetto sviluppa un **agente intelligente** capace di apprendere autonomamente la guida di un kart tramite **Reinforcement Learning (RL)**. Gli obiettivi principali includono:

- Rimanere sulla pista evitando di uscire fuori pista.
- Attraversare tutti i checkpoint in ordine sequenziale.
- Completare il giro nel minor tempo possibile.
- Ottimizzare la traiettoria per massimizzare la ricompensa cumulativa.

1.2 Tecnologie Utilizzate

Componente	**Librerie**
Framework RL	Gymnasium (OpenAI Gym)
Deep Learning	PyTorch
Rendering	Pygame
Analisi Dati	NumPy, Pandas, Matplotlib, Seaborn
Generazione Report	FPDF
Gestione Progetto	UV (Python)

2 Background Teorico

2.1 Reinforcement Learning

Il **Reinforcement Learning** è un paradigma di apprendimento automatico in cui un agente impara a prendere decisioni interagendo con un ambiente. L'agente:

1. **Osserva** lo stato corrente dell'ambiente (s_t)
2. **Seleziona** un'azione (a_t) secondo una policy π
3. **Esegue** l'azione e riceve una ricompensa (r_t)
4. **Transita** in un nuovo stato (s_{t+1})

L'obiettivo è trovare la policy ottimale π^* che massimizza la ricompensa cumulativa attesa.

2.2 Deep Q-Learning

L'utilizzo delle reti neurali, che nel contesto del Q-Learning prendono il nome di Deep Q Network (DQN), cambia completamente la struttura dell'algoritmo del Q-Learning. Per comprendere al meglio come funziona il Deep Q-Learning, dividiamo l'algoritmo in fasi:

1. La prima cosa da fare è creare due DQN: la Policy Network e la Target Network. La Policy Network, o Training Network, è la rete sulla quale viene effettuato l'addestramento ed è aggiornata attivamente durante la fase di apprendimento; viene utilizzata per predire i Q-values per lo stato corrente quando l'agente ha scelto un'azione. La Target Network è la rete utilizzata per calcolare i target Q-values per lo stato successivo durante la fase di apprendimento. Questa rete viene aggiornata meno frequentemente per aumentare la stabilità dell'apprendimento. Il motivo per cui nel Deep Q-Learning si utilizzano due DQN diverse è che esse permettono all'algoritmo di convergere più rapidamente e in modo più stabile. Per comprendere meglio il ruolo di ciascuna rete, osserviamo i passaggi seguenti.

2. Dopo aver creato le due reti, il passo successivo è copiare la Policy Network nella Target Network. Questo passaggio è necessario per garantire coesione tra le due reti all'inizio dell'algoritmo.

3. A questo punto, l'agente sceglie ed esegue l'azione successiva.

4. Inserire lo stato corrente come input nella Policy Network. Si noti che all'inizio dell'addestramento i Q-values in uscita dalla rete saranno numeri casuali, poiché le reti neurali vengono inizializzate con valori casuali per pesi e bias.

5. Inserire lo stato corrente come input nella Target Network per ottenere i Q-values in uscita.

6. Calcolare il Q-value per lo stato corrente e per l'azione scelta dall'agente. Per calcolare il Q-value, se lo stato successivo non è uno stato terminale, viene utilizzata l'equazione di Bellman; altrimenti, il Q-value è uguale alla ricompensa associata a quell'azione.

7. Il Q-value calcolato viene impostato come Q-value dell'azione scelta dall'agente nel livello di output della Target Network.

8. A questo punto, i Q-values della Target Network vengono utilizzati per addestrare la Policy Network. L'addestramento delle reti neurali utilizza la tecnica chiamata backpropagation. Questo processo fa sì che la Policy Network modifichi pesi e bias per avvicinarsi gradualmente ai Q-values target.

9. Ripetere i punti da 3 a 8 fino a quando non viene raggiunto uno stato terminale e l'addestramento termina.

10. Periodicamente, dopo un numero arbitrario di iterazioni, la Target Network viene sincronizzata con la Policy Network. Questo passaggio è cruciale perché consente di evitare instabilità e divergenza durante l'apprendimento. Esistono diversi modi per sincronizzare le due reti, ma il più intuitivo e utilizzato consiste nel copiare la Policy Network nella Target Network.

2.3 Processo di Decisione di Markov (MDP)

L'ambiente è formalizzato come un Markov Decision Process definito dalla tupla

$$(S, A, P, R, \gamma)$$

- S: L'ambiente è formalizzato come un Markov Decision Process definito dalla tupla
- A: Spazio delle azioni (4 direzioni cardinali)
- P: Funzione di transizione probabilistica
- R: Funzione di ricompensa
- γ: Fattore di sconto

3 Struttura del progetto

```
ProgettoAI/
├── src/ ..................................... Codice sorgente
│   ├── main.py ........................... Entry point principale
│   ├── main_costants.py ............ Costanti globali del training
│   ├── agents/ ............................ Logica degli agenti
│   │   ├── agent.py ................. Classi Agent e ReplayBuffer
│   │   ├── agent_costants.py .......... Iperparametri dell'agente
│   │   └── network.py ............ Architettura della Rete Neurale
│   ├── env/ ............................ Ambiente di simulazione
│   │   ├── track_env.py ........... Ambiente Gymnasium Custom
│   │   ├── track_costants.py ............ Costanti dell'ambiente
│   │   └── track_utils.py ................ Utility per il tracciato
│   └── utils/ ............................ Funzioni ausiliarie
│       ├── __init__.py ............ Device detection (CPU/GPU)
│       ├── visual.py ........................ Generazione grafici
│       └── report_generator.py ......... Generazione report PDF
├── data/ ..................................... Risorse statiche
│   ├── tracks/ ........................... File CSV dei tracciati
│   └── music/ ........................... Audio per il rendering
├── models/ ................................ Modelli addestrati
│   └── checkpoints/ ........................ File .pth salvati
├── results/ ........................ Output (grafici, heatmap)
├── notebooks/ .................. Jupyter notebooks (deprecati)
└── docs/ ..................................... Documentazione
```

4 L'Ambiente: TrackEnv

4.1 Descrizione Generale

L'ambiente **TrackEnv** è implementato come una sottoclasse di **gymnasium.Env**, seguendo lo standard OpenAI Gym. Questo permette la compatibilità con qualsiasi algoritmo di RL standard.

4.2 Rappresentazione del circuito

Il circuito è memorizzato come una **matrice 2D** caricata da un **file CSV**. Ogni cella della matrice ha un valore che rappresenta il tipo di terreno:

Valore	Costante	Significato
0.0	`TRACK_OFFROAD_VALUE`	Fuori pista (erba/muro)
0.1	`TRACK_CURB_VALUE`	Cordolo
0.2	`TRACK_ROAD_VALUE`	Strada
0.3	`TRACK_FINISH_VALUE`	Linea del traguardo
-0.1	`TRACK_UNKNOWN_VALUE`	Zona non visibile

4.3 Generazione Matrice Distanze

La matrice delle distanze è sostanzialmente una matrice "ombra" della matrice rappresentante il circuito, dove i valori delle celle, al posto di rappresenta re le differenti parti del circuito, indicano la distanza di ogni cella dal punto di partenza. La generazione della matrice delle distanze avviene tramite un semplice algoritmo di propagazione. Innanzitutto viene inizializzata la matrice delle distanze come una copia della matrice rappresentante il circuito. In seguito viene assegnato il valore -2 a tutte le celle non appartenenti all'insieme delle celle "fuori-strada", valore che indica che alla cella non è stato assegnato il valore della distanza dal traguardo a quella cella. L'algoritmo continua, definendo l'insieme di celle che saranno utilizzate come il punto iniziale del calcolo delle distanze e che definiscono la "direzione di propagazione" dell'algoritmo. Questo insieme è definito manualmente dallo sviluppatore indicando tra quale delle 4 direzioni ("su", "giù", "destra", "sinistra") rispetto alla linea di partenza la propagazione deve iniziare. Per esempio, se la direzione "destra" è stata utilizzata, le celle immediatamente a destra della linea di partenza, saranno le prime celle dell'algoritmo di calcolo delle distanze.
Un ultima operazione che viene fatta prima di calcolare le distanze è, ovviamente, assegnare alle celle della linea di partenza la distanza 0.
Ora vediamo come funziona il sistema di calcolo delle distanze: viene inizializzato una lista contenente l'insieme delle celle immediatamente dopo la linea di partenza citate prima.
Iterativamente dalla lista, finché questa non è vuota, viene estratto il primo elemento secondo la logica FIFO. Una volta estratto un elemento, per ognuno delle quattro celle immediatamente adiacenti, viene calcolata la distanza come la distanza della cella estratta dalla lista + 1.

Il valore della cella vicina viene cambiato solo se tre condizioni sono verificate:

1. Gli indici della cella vicina esistono nella matrice
2. Il valore di quella cella nella matrice delle distanze è -2 e quindi non ancora è ancora stata calcolata la distanza per quella cella
3. La cella non fa parte del traguardo

Se tutte e tre le condizioni sono vere, allora la distanza è calcolata e la cella vicina è aggiunta in fondo alla lista.
Questo algoritmo fa in modo che il calcolo della distanza si propaghi in ogni direzione del tracciato.

4.4 Generazione Matrice Distanza Centro

Durante le fasi di addestramento dell'agente, è emerso che il modello aveva imparato a percorrere il circuito mantenendo traiettorie esterne e molto larghe per massimizzare il punteggio cumulativo. Per correggere l'agente a stringere le curve, è stata introdotta una **Matrice delle Distanze dal Centro**. La matrice viene generata staticamente all'avvio dell'ambiente. L'algoritmo procede attraverso i seguenti passaggi matematici:

1. **Estrazione delle celle valide**: Il sistema scansiona l'intera griglia del circuito, isolando esclusivamente le coordinate appartenenti al tracciato (asfalto, cordoli, traguardo), ignorando le zone di fuoripista.
2. **Definizione dei limiti**: Vengono individuati i limiti estremi del tracciato trovando le coordinate minime e massime sull'asse X e sull'asse Y. Questo crea un rettangolo virtuale che racchiude l'intero circuito.
3. **Individuazione del Centro**: Calcolando la media tra i punti estremi, viene identificato il centro della mappa (C_x, C_y).
4. **Calcolo della Distanza Euclidea**: Per ogni singola cella del tracciato, viene applicato il Teorema di Pitagora per calcolare la distanza in linea d'aria dal centro geometrico:

$$D = \sqrt{(x - C_x)^2 + (y - C_y)^2}$$

Questa matrice viene integrata nel motore fisico dell'ambiente e, ad ogni singolo step, l'ambiente confronta la distanza dal centro della posizione precedente dell'agente con quella della posizione nuova:

- Se l'agente **stringe la traiettoria**, la nuova distanza è minore della vecchia, generando un premio.
- Se l'agente **allarga la curva**, la nuova distanza è maggiore, generando una penalità.

Per evitare che l'agente modifichi il suo comportamento puntando dritto verso il centro ignorando la gara, questo differenza si moltiplica per una costante di smorzamento. In questo modo, l'incentivo principale dell'agente rimane l'avanzamento lungo la pista (calcolato tramite la mappa di distanza dal traguardo), ma a parità di avanzamento, la rete neurale riceve una spinta che premia sistematicamente le traiettorie più interne e performanti. Il risultato finale è una guida molto più realistica, aggressiva ed efficiente.

4.5 Spazio delle Osservazioni

L'agente osserva una **finestra locale** della pista centrata sulla sua posizione. Questa è una matrice quadrata di dimensione **VIEW_SIZE** $\times$ **VIEW_SIZE** (default: 16x16 = 256 celle).
Inoltre, nella modalità di movimento realistica (**velocity**), l'osservazione include anche il vettore della **velocità attuale** del kart.

Listing 1: Definizione dello spazio delle osservazioni.

```
observation_space = gym.spaces.Dict({
    "agent_view": gym.spaces.Box(
        low=TRACK_UNKNOWN_VALUE,
        high=TRACK_FINISH_VALUE,
        shape=(VIEW_SIZE, VIEW_SIZE),
        dtype=np.float32
    ),
    "speed": gym.spaces.Box( # Presente solo in modalita '
       velocity'
        low=-VELOCITY_MAX_SPEED,
        high=VELOCITY_MAX_SPEED,
        shape=(2,),
        dtype=np.int32
    )
})
```

Questa osservazione locale permette all'agente di avere una **percezione limitata** della pista, simile a una vista in prima persona.

4.6 Spazio delle azioni

L'ambiente supporta due modalità di movimento, configurabili tramite la costante **MOVEMENT_MODE**.

4.6.1 Modalità "velocity" (Accelerazione)

In questa modalità più realistica, l'agente controlla l'accelerazione del kart anziché la posizione assoluta. La velocità viene cumulata ad ogni step e la nuova posizione è calcolata in base alla velocità risultante.
L'agente può scegliere tra **9 azioni discrete** che rappresentano le combinazioni di accelerazione sugli assi:

```
action_space = gym.spaces.Discrete(9) #VELOCITY_ACTION_SPACE_SIZE
```

Questa modalità include anche controlli per verificare se la traiettoria percorsa interseca zone fuori pista, rendendo la simulazione più fisica e prevenendo comportamenti non permessi ("salti" sopra l'erba).

Azione	Accelerazione (Riga, Colonna)	Descrizione
0	(-1, -1)	Alto-Sinistra
1	(-1, 0)	Alto
2	(-1, 1)	Alto-Destra
3	(0, -1)	Sinistra
4	(0, 0)	Nessuna Accelerazione
5	(0, 1)	Destra
6	(1, -1)	Basso-Sinistra
7	(1, 0)	Basso
8	(1, 1)	Basso-Destra

4.6.2 Modalità "simple" (Movimento Ridotto)

Nella modalità classica, l'agente sceglie semplicemente di spostarsi di una singola cella nelle **4 direzioni cardinali**:

Azione	Direzione	Vettore Movimento
0	Destra	(0, +1)
1	Alto	(-1, 0)
2	Sinistra	(0, -1)
3	Basso	(+1, 0)

```
action_space = gym.spaces.Discrete(4)
```

4.7 Checkpoints

Il sistema di ricompensa dell'ambiente si basa sulla presenza di una serie di checkpoint lungo il circuito. Questi checkpoint non sono altro che liste di celle della matrice circuito.

Ogni checkpoint è costruito in funzione della sua distanza dalla linea di partenza, infatti le celle appartenenti ad un checkpoint sono tutte quelle celle che hanno la stessa distanza dal traguardo. Tutti i checkpoint sono costruiti in modo dinamico alla matrice delle distanze base.

Infatti, dato un numero arbitrario di checkpoint che si vuole avere nella pista, vengono prima definite le distanze a cui i checkpoint devono essere dal traguardo per essere distribuiti in modo equidistante gli uni dagl'altri. In seguito vengo creati i checkpoint in sé, delle liste in cui vengono inserite tutte le celle della matrice circuito che hanno la distanza pari alla distanza definita per quel determinato checkpoint.

Nella fase di training, quando l'agente passa per un checkpoint, viene ricompensato in modo molto positivo, però per evitare che l'agente cerchi di compiere comportamenti e strategie 'illegali', come andare direttamente al traguardo senza passare per i checkpoint, è stato implementato un sistema di progresso.

Infatti, il traguardo e i checkpoint non sono sempre attivi, ma sono attivati sequenzialmente uno dopo l'altro. Per esempio il terzo checkpoint non sarà attivo se l'agente non ha attraversato il secondo, e così via. Il traguardo è attivo soltanto quando l'agente ha attraversato tutti i checkpoint.

Ovviamente il progresso dell'agente viene resettato ad ogni episodio.

4.8 Sistema di Ricompense

La funzione di reward è usata per guidare l'apprendimento dell'agente. E' stata progettata per:

- **Incentivare** il progresso verso il traguardo
- **Scegliere** il percorso più breve
- **Penalizzare** comportamenti indesiderati
- **Evitare** loop infiniti o stalli

Tabella dei Reward

Evento	Valore	Descrizione
`FINISH_REWARD`	+100	Completamento giro (tutti i checkpoint superati)
`CHECKPOINT_REWARD`	+30	Attraversamento checkpoint
`OFFROAD_REWARD`	-50	Uscita di pista (termina episodio)
`STEP_PENALTY`	-0.01	Penalità per ogni step (incentiva velocità)
`REPEAT_PENALTY`	-0.5	Penalità per visitare celle già visitate
`BACKWARD_PENALTY`	$\times 1$	Moltiplicatore per movimento all'indietro
Progresso positivo	+delta	Ricompensa proporzionale all'avanzamento
Prima visita cella	+0.1	Bonus per esplorazione

4.9 Rendering con PyGame

L'ambiente include un sistema di rendering visivo in stile **gioco arcade** che permette di visualizzare:

- **Il circuito** con erba, strada, cordoli e traguardo
- **La macchina** con animazioni di rotazione
- **I checkpoint** rappresentati come stelle
- **Decorazioni** come la griglia e lo sfondo sfumato
- **HUD** con progressi checkpoint e tempo

5 L'Agente: Deep Q-Network

5.1 Architettura della Rete Neurale

La rete neurale Network è un **Multi-Layer Perceptron (MLP)** che prende in input l'osservazione appiattita e produce i Q-values per ogni azione.

Implementazione PyTorch

Listing 2: Architettura della Deep Q-Network

```
class Network(nn.Module):
    def __init__(self, input_dim, action_size, seed=42):
        super(Network, self).__init__()
        self.seed = torch.manual_seed(seed)

        # Strati lineari (Fully Connected)
        self.fc1 = nn.Linear(input_dim, 256)    # Input -> 256
        self.fc2 = nn.Linear(256, 256)          # 256 -> 256
        self.fc3 = nn.Linear(256, action_size) # 256 -> Azioni

    def forward(self, x):
        # Mappa lo stato ai Q-values per ogni azione.
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)
```

5.2 Experience Replay

Un aspetto fondamentale dell'addestramento di una rete neurale è quello di addestrarla con dati di addestramento randomizzati.

Questo è molto importante perché elimina la correlazione temporale tra i campioni, consentendo alla rete neurale di generalizzare più facilmente e di non apprendere una situazione molto specifica.

Per fare questo, nel Deep Q Learning, viene utilizzato l'experience replay. L'experience replay consiste nell'utilizzare un array per memorizzare la memoria dell'agente. Questa memoria ha una lunghezza fissa e ogni volta che si riempie, l'ultimo elemento, ovvero la memoria più vecchia, viene eliminato dall'array.

Quando l'agente sceglie un'azione e la compie (fase 3), invece di passare lo stato direttamente alla Policy Network, memorizziamo i dati di questa interazione tra l'agente e l'ambiente (la tupla stato, azione, stato successivo, ricompensa) nella memoria.

Successivamente, un numero arbitrario di memorie viene campionato in modo casuale dalla memoria e passato come dati di addestramento per la Policy Network (fase 4).

A seguito viene riportata la definizione della classe ReplayBuffer, la quale implementa la memoria dell'experience replay:

Listing 3: Implementazione del Replay Buffer

```python
class ReplayBuffer:
    def __init__(self, capacity, batch_size, seed=42):
        self.memory = deque(maxlen=capacity)
        self.batch_size = batch_size

    def add(self, state, action, reward, next_state, done):
        # Aggiunge una transizione alla memoria.
        self.memory.append((state, action, reward, next_state,
           done))

    def sample(self):
        # Campiona casualmente un batch di esperienze.
        batch = random.sample(self.memory, self.batch_size)
        return zip(*batch)
```

5.3 Iperparametri dell'Agente

Parametro	Valore	Descrizione
LEARNING_RATE	0.0001	Tasso di apprendimento
MINIBATCH_SIZE	100	Dimensione batch per training
REPLAY_BUFFER_SIZE	200,000	Capacità memoria replay
DISCOUNT_FACTOR (γ)	0.99	Peso ricompense future
EPSILON_START	1.0	Epsilon iniziale
EPSILON_MIN	0.01	Epsilon minimo finale
EPSILON_DECAY_PORTION	0.8	Porzione episodi per decay
TARGET_UPDATE_EVERY	4	Frequenza aggiornamento della rete

5.4 Strategia Epsilon-Greedy

L'esplorazione è gestita tramite una strategia epsilon-greedy con decadimento. Sono supportati due tipi di decay:

1. Decadimento lineare
2. Decadimento esponenziale (Default)

Listing 4: Logica di decadimento di epsilon

```python
def update_epsilon(self):
    # Gestisce il passaggio dall'esplorazione allo sfruttamento.
    match eps_type:
        case "lineare":
            self.epsilon = max(epsilon_min, epsilon - decay)
        case "esponenziale":
            self.epsilon = max(epsilon_min, epsilon * decay)
```

5.5 Algoritmo di Apprendimento

La funzione learn() implementa l'aggiornamento dei pesi della Q-network usando la **Temporal Difference (TD) loss**:

Listing 5: Implementazione dell'aggiornamento TD nella DQN

```python
def learn(self):
    # Esegue l'aggiornamento dei pesi tramite backpropagation.
    # Campiona mini-batch dal replay buffer
    states, actions, rewards, next_states, dones = self.memory.
        sample()

    # Calcola Q-values previsti
    q_expected = self.q_net(states).gather(1, actions)

    # Calcola Q-target usando la target network
    with torch.no_grad():
        q_next = self.target_net(next_states).max(1)[0]
        q_target = rewards + (gamma * q_next * (1 - dones))

    # Calcola loss MSE e aggiorna pesi
    loss = F.mse_loss(q_expected, q_target)

    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
```

6 Pipeline di Training

6.1 Flusso di esecuzione

Il progetto viene lanciato col il comando: **python src/main.py train** . C'é la possibilità di variare il numero di episodi usati per il training col l'apposita estensione –**ep NUM_EPISODI**.

1. Inizializzazione:
   - Crea l'ambiente TrackEnv
   - Inizializza l'agente con Q-network e target network
   - Crea il replay buffer vuoto
2. Loop degli Episodi:

Listing 6: Ciclo principale di addestramento per episodio

```
for episodio in 1..DEFAULT_TRAIN_EPISODES:
    state = env.reset()

    while not done:
        # Selezione ed esecuzione dell'azione
        action = agent.select_action(state)
        next_state, reward, done = env.step(action)

        # Salvataggio dell'esperienza e apprendimento
        agent.step(state, action, reward, next_state,
            done)
        state = next_state

    # Aggiornamento parametri post-episodio
    agent.update_epsilon()  # Decadimento di epsilon

    # Salvataggio del miglior modello e dei
       checkpoint
    if is_best_average:
        save_model("best_model.pth")

    if episodio % SAVE_CHECKPOINT_EVERY == 0:
        save_checkpoint(f"cp_{episodio}.pth")
```

3. Output Finale:
   - Grafico andamento reward
   - Heatmap delle traiettorie
   - Report PDF

6.2 Parametri di Training

Parametro	Valore	Descrizione
`DEFAULT_TRAIN_EPISODES`	100'000	Numero episodi default
`STEP_LIMIT`	1000	Step massimi per episodio
`SCORES_WINDOW_SIZE`	1000	Finestra media mobile
`SAVE_CHECKPOINT_EVERY`	1000	Frequenza salvataggio

7 Testing e Valutazione

7.1 Modalità Test

Esecuzione del testo con **python src/main.py test**.
C'é la possibilità di cambiare il modello usato col l'apposita estensione **–mod cp_XXXX.pth**

In modalità test:

- Epsilon è impostato a 0 (non esplora, ma fa quello che sa)
- Il rendering è attivo per visualizzare l'agente

7.2 Metriche di Valutazione

1. **Score Medio**: Media mobile dei reward totali per episodio
2. **Heatmap Traiettorie**: Colora ogni cella del tracciato in base al numero di volte in cui è stata visitata durante il training
3. **Completamento Giro**: Percentuale di episodi che raggiungono il traguardo
4. **Tempo per Checkpoint**: Step medi per raggiungere ogni checkpoint

8 Circuiti Disponibili

I nomi dei circuiti si ispirano ai circuiti moderni usati in Formula 1, ma la loro struttura usata in questo progetto non ha nulla a che vedere con la loro controparte reale. Tutti i tracciati hanno una grandezza di 60x60 caselle, ed una larghezza di pista di circa 7 celle (a seconda del tracciato di riferimento).
L'unico circuito utilizzato in fase di training e di testing è il circuito denominato come Circuito di Imola.

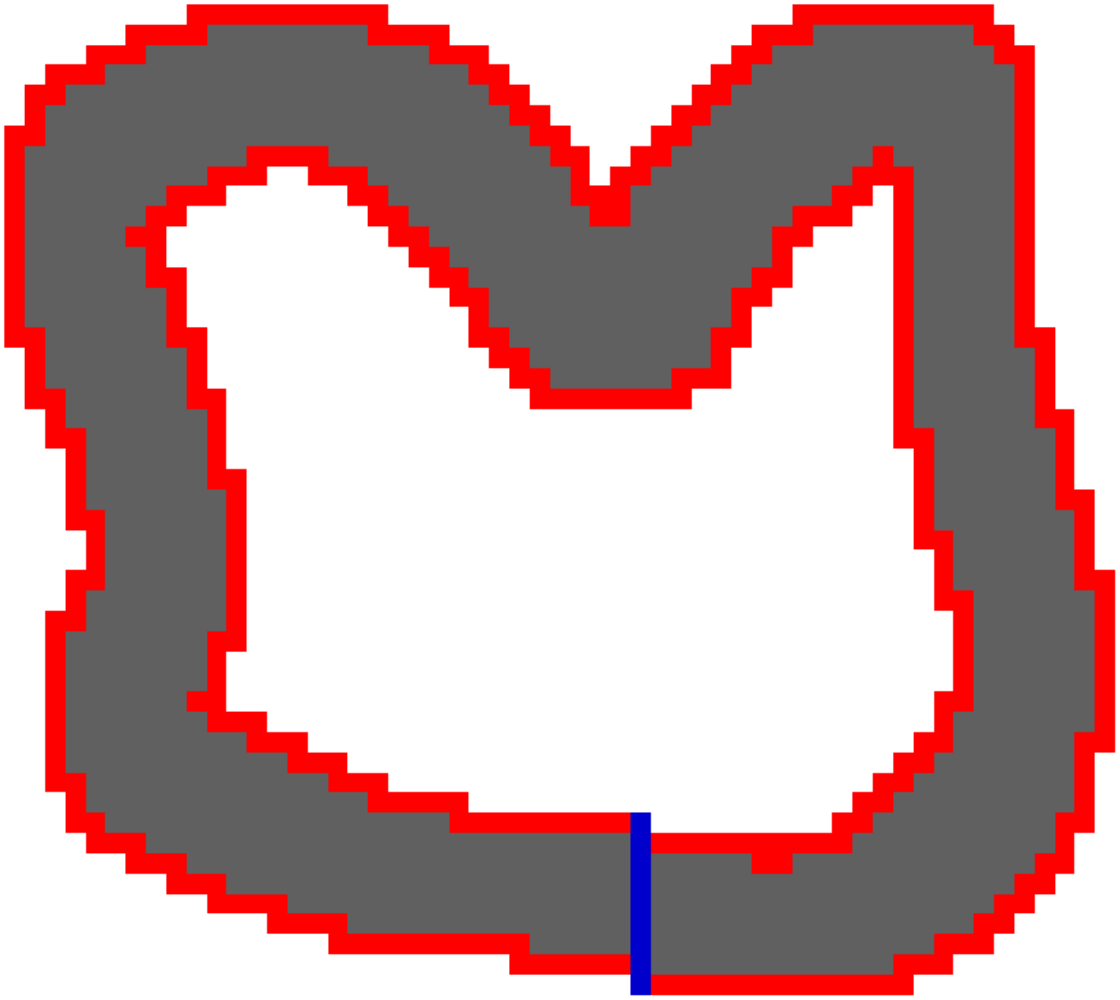
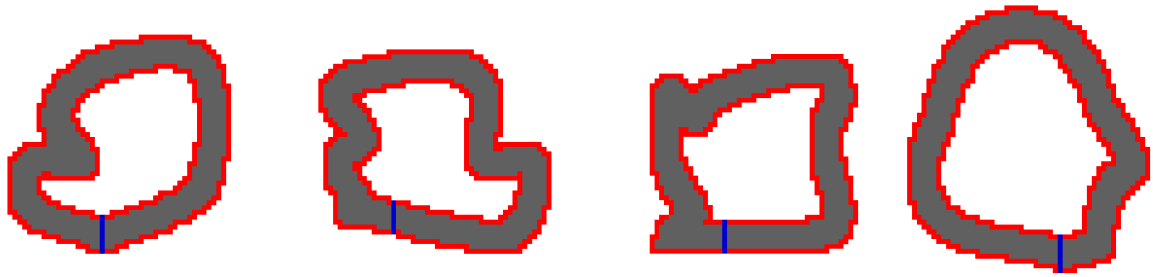


Figura 1: Circuito Imola

8.1 Altri tracciati

Altri tracciati sono stati inclusi nel progetto, lasciando la possibilità di testare ed estendere il progetto in futuro. Segue la lista dei tracciati non utilizzati:



Circuito Austria | Circuito Brasile | Circuito Monza | Circuito SPA

Figura 2: Ulteriori circuiti disponibili

9 Colonna Sonora

Il progetto supporta la musica di sottofondo durante il testing del progetto. Per motivi di **copyright** non è stata inclusa nel repository.
Nel caso si voglia usare, bisogna mettere un file di tipo *mp3* con nome "coconut_mall" nella cartella *data/music/*

NOTA: Nomi o riferimenti presenti in questa colonna sonora sono utilizzati esclusivamente a scopo di intrattenimento. Eventuali somiglianze con opere o marchi esistenti sono casuali.

10 Strumenti di Analisi

10.1 Grafico Training

La funzione save_training_plot() genera un grafico PNG dell'andamento del training:

- Asse X: Numero di episodio
- Asse Y: Reward totale (score)
- **Curva**: Media mobile su 1000 episodi

10.2 Heatmap

Il sistema genera due tipi di heatmap:

1. **heatmap.csv**: Matrice con conteggio visite per cella.
2. **heatmap.png**: Visualizzazione con scala logaritmica colorata

La scala logaritmica permette di visualizzare sia celle poco visitate che aree ad alta densità.

10.3 Report PDF

Al termine del training viene generato un report PDF contenente:

1. **Parametri Agente**: Tutti gli iperparametri usati
2. **Parametri Pista**: Configurazione dell'ambiente
3. **Architettura Rete**: Layer e dimensioni
4. Grafico Training: Andamento dei reward
5. **Heatmap**: Traiettorie percorse
6. **Codice Sorgente**: Funzione step() completa

11 Ottimizzazioni

1. **Reward Shaping Avanzato:**
   - Bonus per esplorazione di nuove celle
   - Penalità crescente per celle rivisitate
   - Terminazione episodio in caso di oscillazione del modello
2. **Sistema Checkpoint:**
   - Impedisce le scorciatoie
   - Fornisce reward intermedi utili al progresso
3. **Supporto GPU:**
   - Rilevamento automatico CUDA
   - Trasferimento tensori su device appropriato

12 Grafica del Tracciato

Per dare un aspetto più intuitivo al modello durante la fase di testing, nell'ambiente è stato incluso del codice che ne migliora l'esperienza visiva.

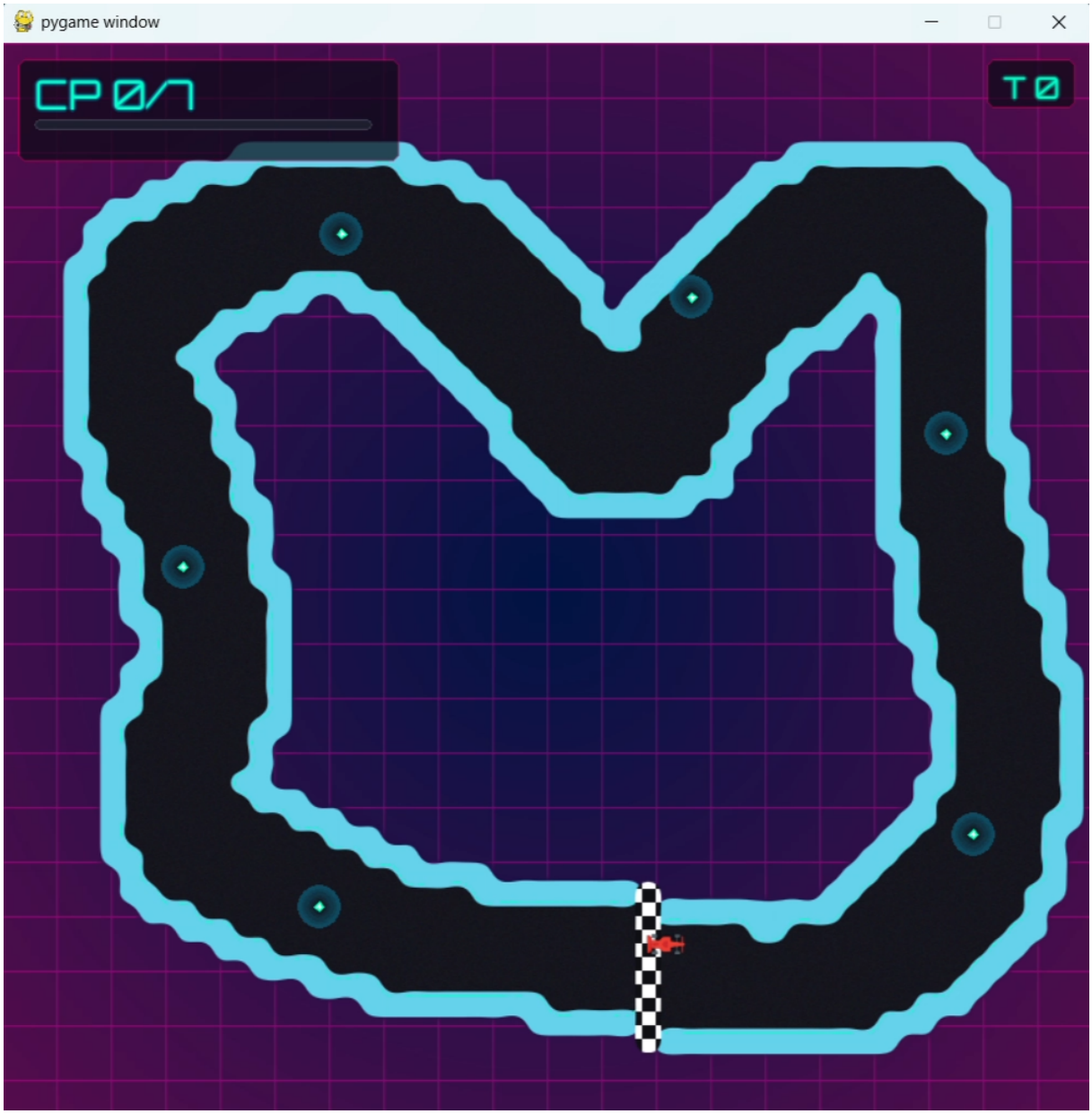


Figura 3: Interfaccia grafica rappresentate l'agente sul circuito durante il test, con progresso checkpoint e numero di step

13 Risultati

#	Esperimento	Modalità	Episodi	Computazione	Tempo
1	Agente base	Simple (4 azioni)	200'000	RTX 3060 12 GB	4 ore 11 min
2	Ambiente migliorato	Simple (4 azioni)	100'000	RTX 5070	2 ore
3	Agente con accelerazione	Velocity (9 azioni)	100'000	RTX 5070	50 minuti

13.1 Esperimento 1: Agente Base (Movimento Discreto)

Questo esperimento (prima versione) ha usato un ambiente che usa solamente la matrice delle distanze base. Tale addestramento, a differenza di quelli successivi, ha richiesto l'utilizzo di 200'000 episodi, in quanto non in grado di apprendere la percorrenza di un giro completo in meno tentativi.

Analisi del Grafico

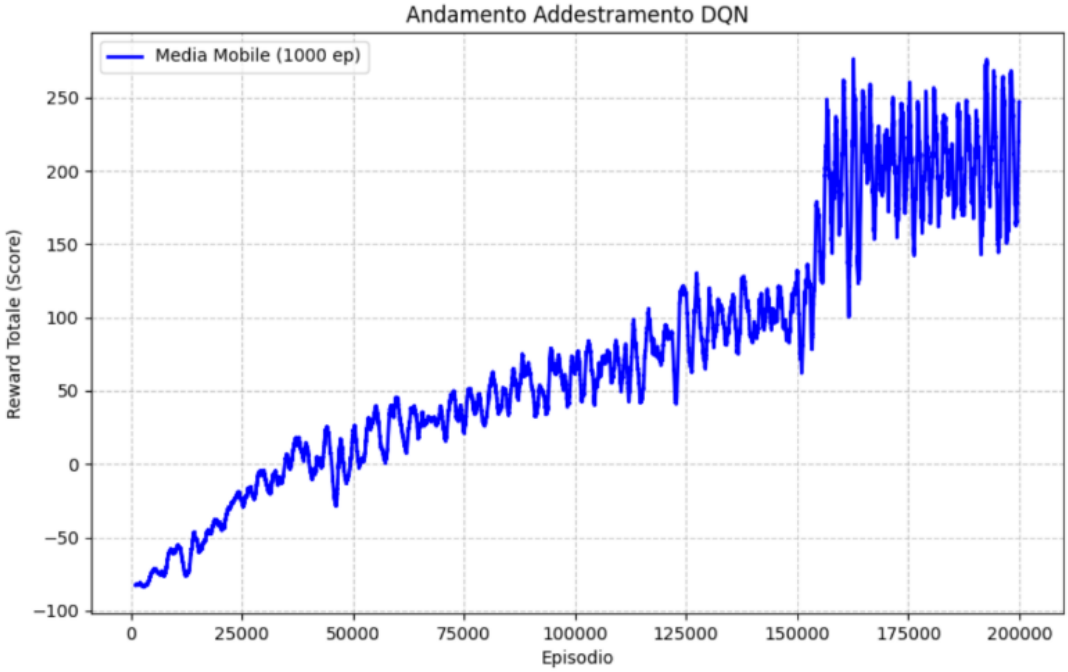


Figura 4: Andemento del modello con ambiente base, con media mobile di 1000 episodi.

Il grafico mostra una crescita altalenante ma costante, fino a 150'000 episodi circa, dove si verifica un exploit con visibile aumento del reward medio. Successivamente, raggiunge un plateau, dopo di ché comincia ad essere instabile per i restanti 50'000 episodi.

Analisi della Heatmap

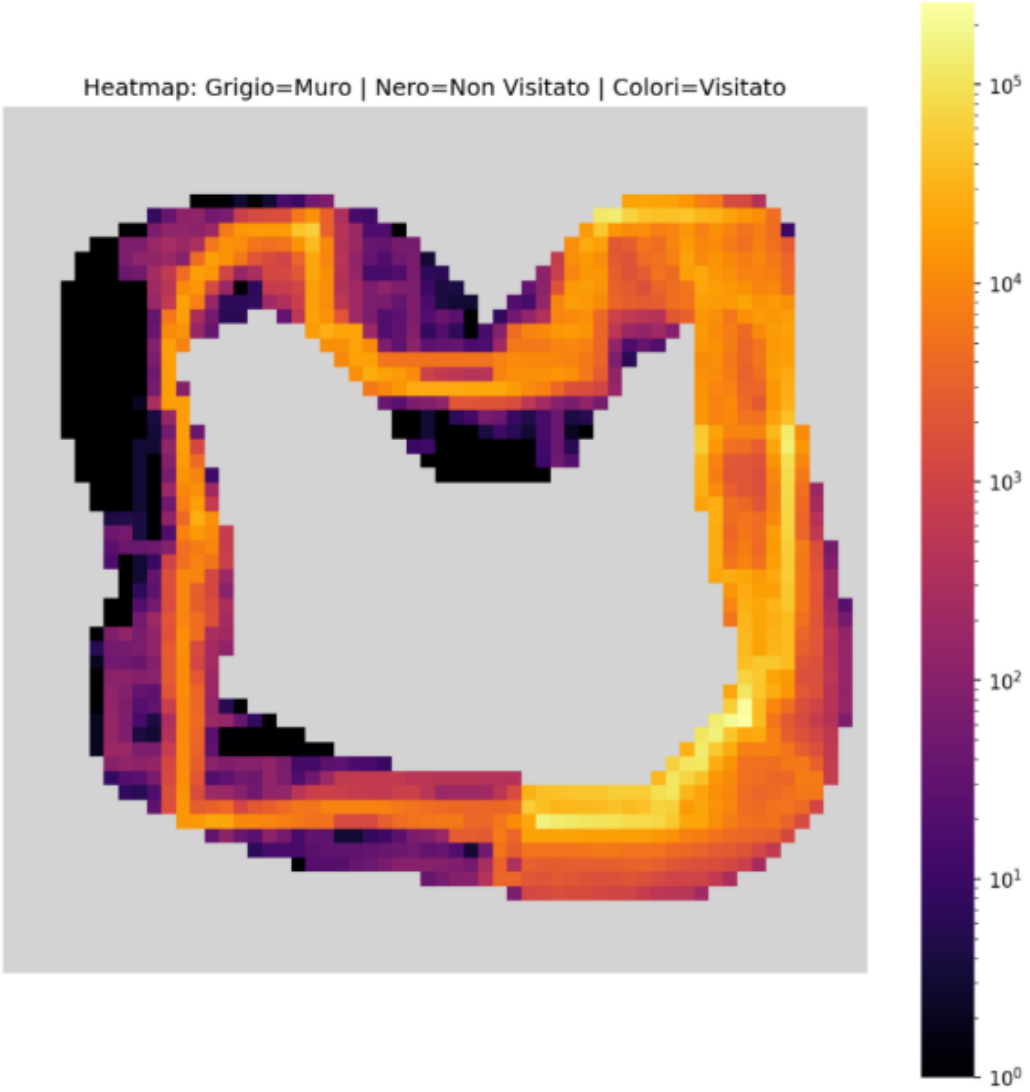


Figura 5: Heatmap del modello con ambiente base, con inclusa la legenda che indica il numero di volte in cui il modello passa su una certa cella

Il modello esegue una fase di esplorazione, portando a visitare gran parte dello spazio percorribile della prima metà del circuito, data dalla forte influenza della componente casuale epsilon. Successivamente, l'agente percorre una traiettoria sub-ottimale, con visibile diminuzione dell'esplorazione, lasciando dedurre l'uso di una logica più "razionale" nel prendere decisioni.

13.2 Esperimento 2: Ambiente Migliorato

Questo esperimento ha usato un ambiente che usa una matrice ausiliaria, in aggiunta alla matrice delle distanze utilizzata dall'ambiente base.

Analisi del Grafico

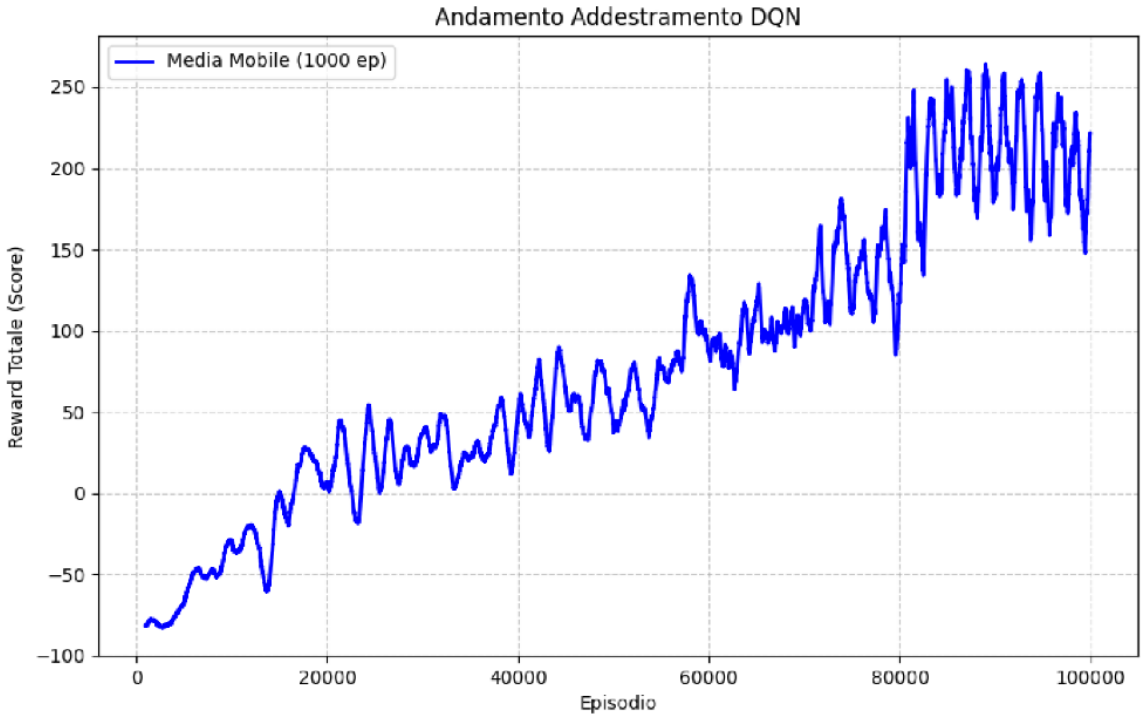


Figura 6: Andemento del modello con ambiente migliorato, con media mobile di 1000 episodi.

Il grafico mostra una crescita altalenante ma costante, fino agli 80'000 episodi dove raggiunge un plateau, dopo di ché comincia ad essere instabile per i restanti 20'000 episodi. Il comportamento è analogo all'esperimento precedente.

Analisi della Heatmap

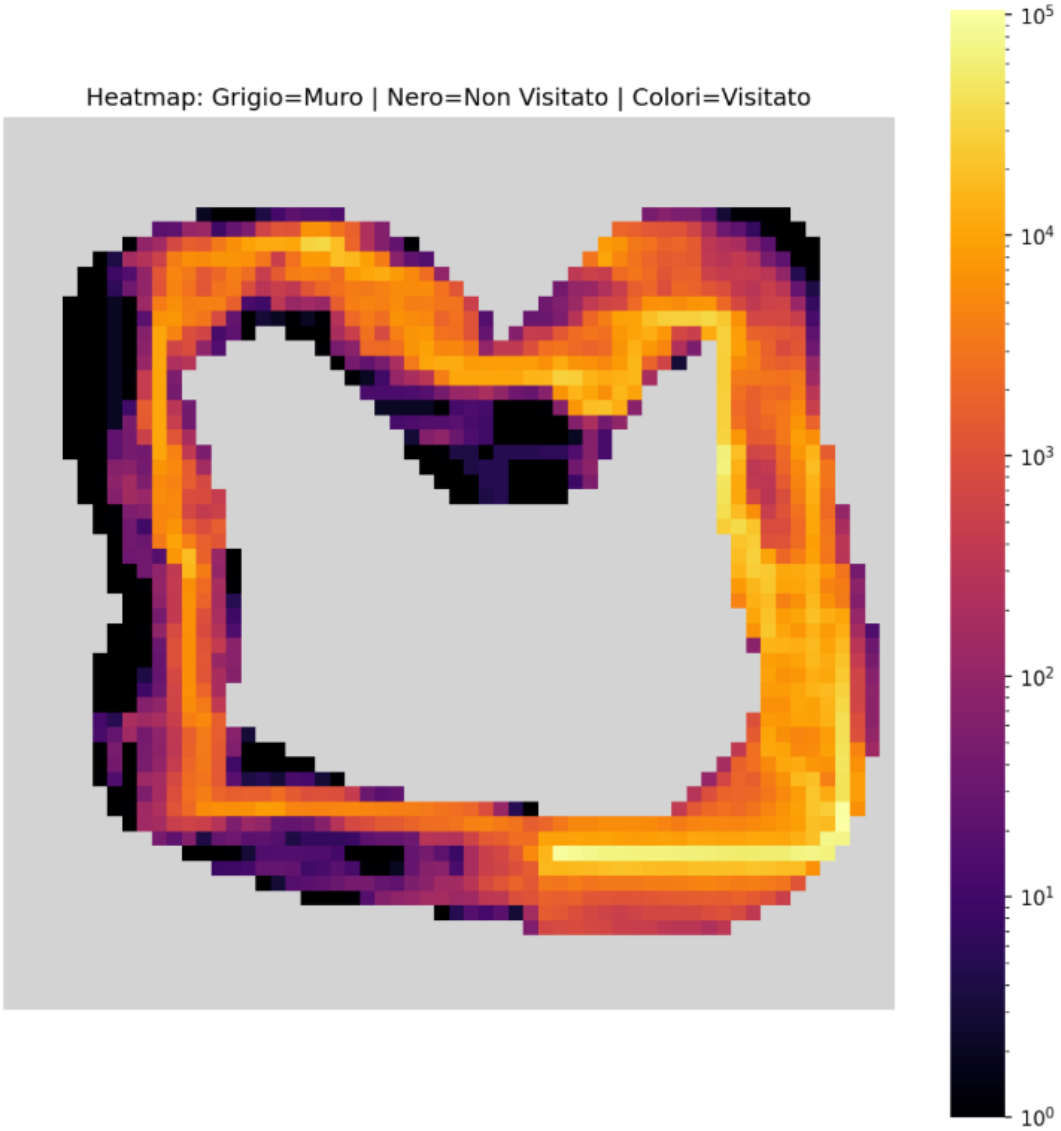


Figura 7: Heatmap del modello con ambiente migliorato, con inclusa la legenda che indica il numero di volte in cui il modello passa su una certa cella

A parte una prima fase di esplorazione del tracciato, dopo le prime curve il modello impara una traiettoria che sub-ottimale per il circuito su cui è stato allenato.

13.3 Esperimento 3: Agente con Accelerazione

Questo esperimento ha usato un modello capace di accelerare e frenare, con conseguenti più azioni rispetto al modello base, aumentandone la complessità.
Rispetto agli altri esperimenti, questo modello durante la fase di training

non conta esattamente i **checkpoint**, che forniscono reward positivo che ne aumentano i valori nel grafico. Questa scelta è stata fatta in quanto si è visto che il modello apprendeva meglio se i checkpoint non fossero calcolati in modo rigoroso.
Più precisamente, il modello salta più celle alla volta grazie alla sua accelerazione; nel caso superi un checkpoint durante il suo salto, esso non viene conteggiato. A primo impatto può sembrare una svista logica, ma dopo una serie di test, si è dimostrata una scelta corretta.

Modifiche rispetto ai parametri di Default

Parametro	Valore Precedente	Valore Utilizzato
`DISCOUNT_FACTOR` (γ)	0.99	0.9
`BACKWARD_PENALTY`	1	2

Analisi del Grafico

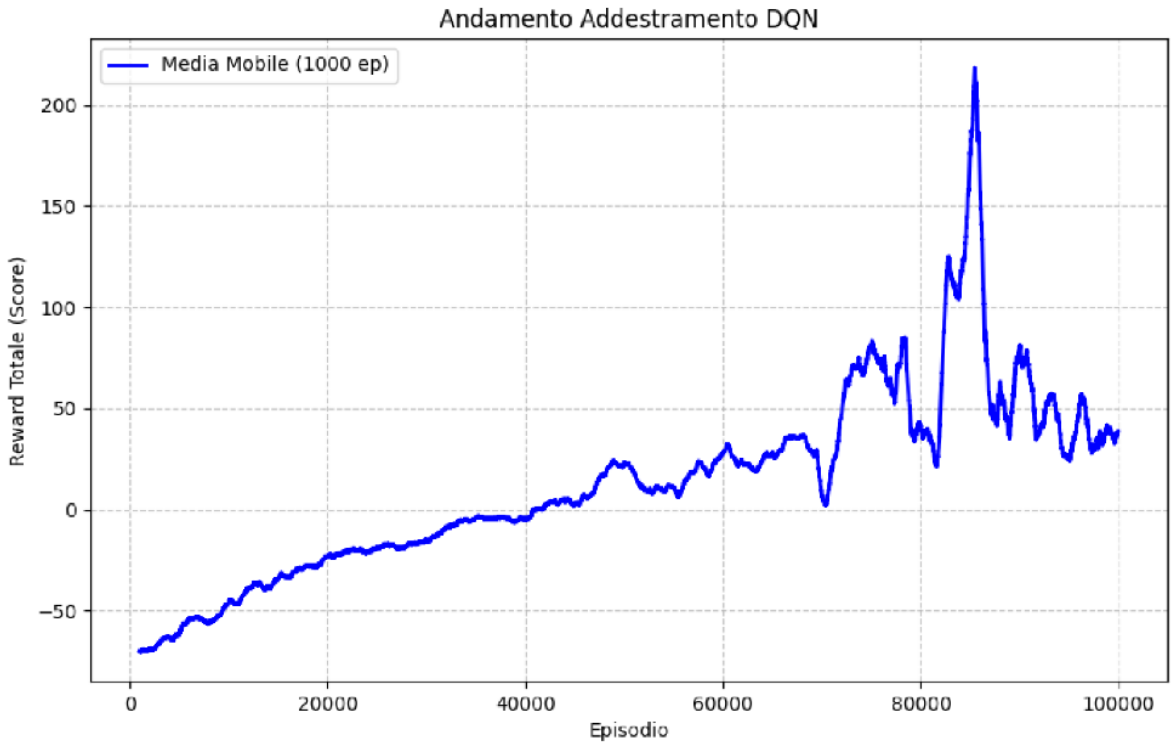


Figura 8: Andemento del modello con accelerazione, con media mobile di 1000 episodi.

Il grafico mostra una crescita quasi lineare fino a circa 70'000 episodi, per poi diventare più instabile ma guadagnare anche più reward, con un picco anomalo di **220** di reward.

Analisi della Heatmap

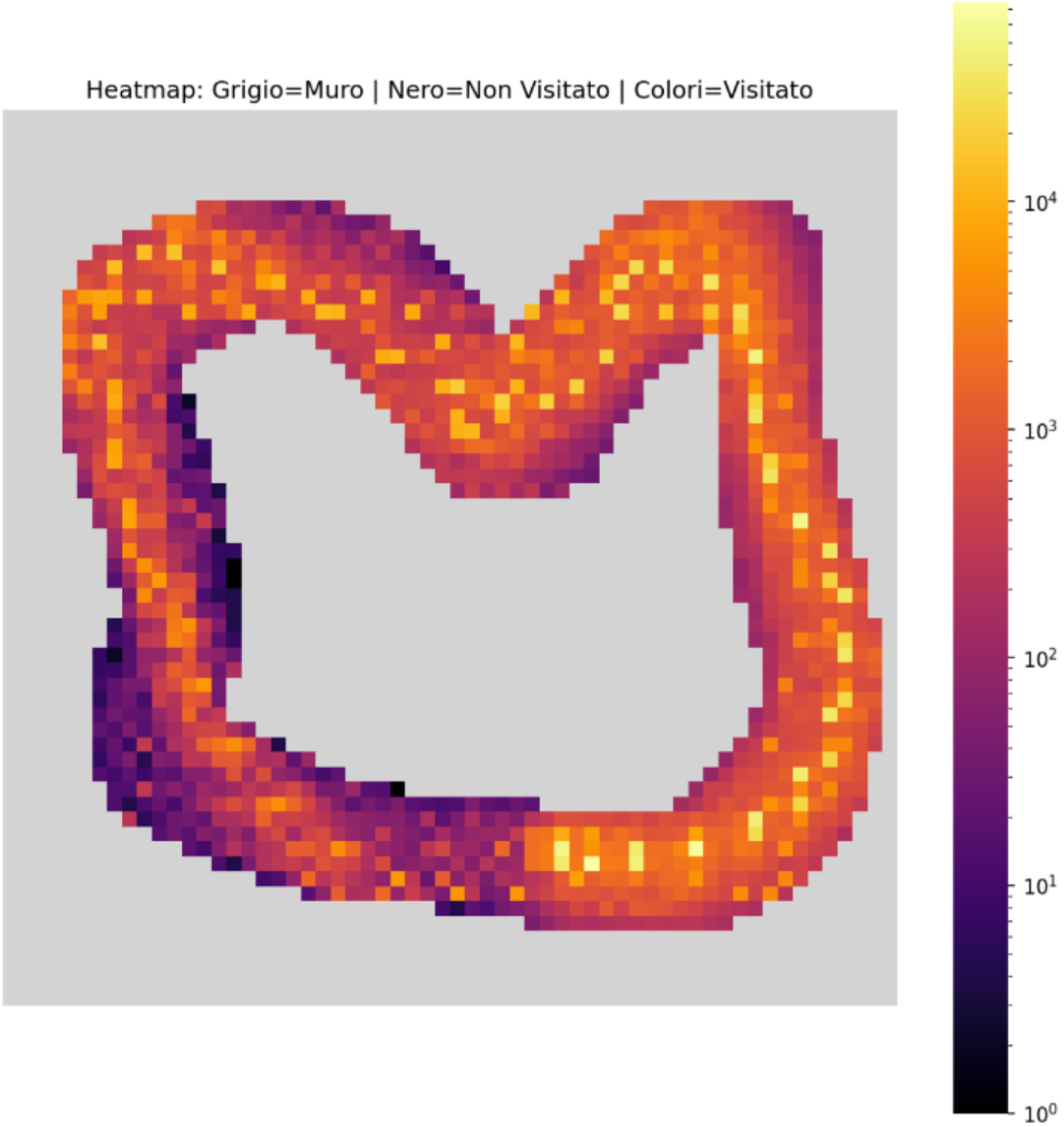


Figura 9: Heatmap del modello con accelerazione, con inclusa la legenda che indica il numero di volte in cui il modello passa su una certa cella.

Dalla Heatmap si può vedere il modello asplora praticamente tutta la pista, e con 100'000 episodi trova una strategia sub-ottimale per il 75% della pista. Un training con più episodi potrebbe portare questo valore a 100%, ma già con 100'000 episodi si ottengono solidi risultati.

14 Conclusioni e lavori futuri

Osservando le diverse versioni dell'ambiente mostrate sopra, possiamo trarre diverse conclusioni. La prima riguarda il modo con cui l'agente esplora

l'ambiente nel corso della fase di addestramento. Nella versione base dell'ambiente, l'agente non riesce ad esplorare in modo esaustivo l'interezza dell'ambiente, come mostrato nella heatmap 13.1. Lo stesso è vero per quanto riguarda il modello dell'ambiente migliorato con l'utilizzo della matrice ausiliaria, il quale però presenta un apprendimento più veloce rispetto alla sua versione base. Infine, la versione dell'ambiente basata sull'accelerazione, è l'unica che permette all'agente di esplorare in modo uniforme e completo l'interezza dell'ambiente. Questo potrebbe essere uno dei fattori che porta alla forma diversa della curva di apprendimento rispetto a quella dei modelli precedenti.

Un'altra considerazione interessante è la differenza nella stabilità dell'apprendimento delle diverse versioni dell'ambiente. Le prime due versioni mostrano un apprendimento stabile e una curva di apprendimento a dente di sega, mentre la terza versione mostra un apprendimento più instabile e con una curva di apprendimento molto più appiattita rispetto alle altre. Questo può essere causato da diversi fattori; uno è senza dubbio legato a una modifica al conteggio dei checkpoint, come citato nella sezione 13.3. Altri fattori potrebbe essere la differenza in complessità tra le versioni degli ambienti. Nei primi due ambienti, per esempio, lo spazio delle azioni dell'agente è composto soltanto di 4 azioni, mentre lo spaio delle azioni dell'agente nell'ambiente con accelerazione è di 9 azioni. Questa differenza potrebbe sembrare trascurabile ma ha un grandissimo impatto sull'apprendimento dell'agente. Un ambiente più complesso richiede più apprendimento e potrebbe presentare più instabilità, dato il maggior numero di fattori in gioco. In conclusione, le diverse versioni dell'ambiente mostrate sono solo 3 delle infinite possibili implementazioni che cercano di risolvere questo problema. Nonostante ciò, crediamo che queste siano un buon punto di partenza per ideare un sistema che riesca a trovare il vero percorso ottimale.

Come lavori futuri, si potrebbe lavorare più profondamente sulle diverse componenti delle nostre implementazioni. Per esempio, si potrebbe implementare un sistema di simulazione dell'attrito e delle traiettorie più fedele alla realtà; oppure si potrebbe sviluppare una funzione di reward più raffinata e adatta al problema; o ancora si potrebbe provare a cambiare l'algoritmo del Q-Learning con altri come SARSA o modelli Attore-Critico. Infine, in aggiunta alle modifiche sopra citate, si potrebbe far addestrare l'agente su più circuiti così da permettergli di generalizzare il suo comportamento e creare quindi un agente capace di percorrere qualsiasi circuito (come quelli mostrati nella sezione 8.1). Per quanto le nostre implementa-

zioni non siano riuscite a trovare il vero percorso ottimale e non raggiungano lo standard dell'attuale stato dell'arte, crediamo che queste ci abbiano aiutare a comprendere come funziona il vasto mondo del Reinforcement Learning.

15 Repository

Il codice sorgente completo è disponibile nella repository GitHub del progetto al seguente link: `https://github.com/EChiarini/ProgettoAI`